Assignment 9: Build, Stress, Observe, and Explain Memory Pressure

Linux Systems and Performance - Week 9

Watermarks and Reclaim Paths -> PSI, Thrashing, and Swap -> Workload Design and Sysctl Tuning

1. Assignment Philosophy

This assignment is not evaluated only by whether your tools run. You will build small memory-stress and monitoring tools, apply controlled memory pressure, observe how Linux reclaims memory and how tasks are delayed, measure how configuration changes affect the system, and explain the causes of the results.

BUILD -> STRESS -> OBSERVE -> DIAGNOSE -> EXPLAIN

Important: AI coding tools may be used. However, you are responsible for understanding, debugging, modifying, and explaining all submitted code and experimental results. Implementation alone is not sufficient for full credit.

Safety: Run all experiments inside WSL 2 on a laptop you control, using a dedicated lab distribution. WSL distributions share kernel, RAM and swap: a separate distribution alone does not isolate memory experiments. Save work and stop other WSL distributions and Docker Desktop before testing. Use bounded workloads; OOM is not a goal. Stop the workload if the laptop becomes unresponsive. Record failures and recover before continuing.

2. Challenge Levels

Level	Build	Stress / Trigger	Observe / Measure	Main Question
Level 1	memhog (anon, file, probe modes) + counter sampling	file-cache pressure and an anonymous allocation burst	watermarks, pgscan_*, allocstall, allocation latency	Who reclaims memory, and when does a task start to wait?
Level 2	psimon (polling + PSI trigger)	CPU contention, file thrashing, disk swap-in; optional zram	PSI some/full, refault and swap counters	How do we tell what a task is waiting for?
Level 3	C workload design + psimon and sampler	one-at-a-time sysctl changes under a purpose-designed workload	PSI, probe latency, workload performance, reclaim and swap rates	How does tuning change behavior for this workload, and at what cost?

Levels are cumulative. Use the same tools, the same sampler, and the same workload definitions across levels so that comparisons are meaningful. Absolute numbers across different machines are not graded.

3. Common Interface and Rules

- Student implementation: memhog and psimon in C, a zoneinfo script in shell or Python, and a C workload extension for Level 3. Shell/Python may automate runs and analyze data. The supplied student_support/sample.py is a read-only helper; you do not implement it. cgrun is not required.

- Use WSL 2 with a Linux 6.x kernel and an Ubuntu distribution. WSL 1 and Docker containers are not supported lab environments. Use your existing WSL 2 installation and Linux administrator access; complete the capability checks in Section 3.1. CPU count and memory limits may differ between students. Record actual resources and any automatic cache-reclaim setting; keep them unchanged throughout each comparison. Record Windows, WSL, distribution and kernel versions, actual memory and swap, and the four VM sysctl values. cgroup implementation is not required; record any existing memory or swap limits affecting your process.

- Define P as idle MemAvailable after environment preparation and before each experiment set. Record P and convert all sizes to bytes. Keep those byte sizes fixed across repeated and baseline/changed runs in that set. Recalibrating from a different P after tuning would change the workload. The suggested fractions are starting points; pilot and document adjustments before collecting the comparison.

- Use the supplied student_support/sample.py unchanged for all required CSV measurements. It records monotonic_s and elapsed_s, memory and CPU PSI, /proc/vmstat, /proc/meminfo, and per-zone free/min/low/high at approximately 1 s intervals. Memory values use KiB; zone values use pages. Parse the quoted zones_json cell with CSV and JSON libraries. Unsupported values are NA, not zero. This collector does not replace the C psimon requirement.

- Report counters as rates: the difference between two samples divided by the elapsed time. /proc/vmstat counters are cumulative and are not meaningful as raw values.

- Before each run, stop lab workloads, record idle state, and apply the same documented cache policy. Use sync and, for cold-cache tests, sudo sh -c "echo 3 > /proc/sys/vm/drop_caches" only after stopping other WSL work. Warm-cache tests must instead perform the same explicit warm-up before measurement. Cache dropping does not reset swap state or cumulative counters. After changing swap/backend, allow the VM to settle before sampling.

- For every comparison, keep the machine, kernel settings, workload definition, and run length fixed. Repeat each configuration at least three times and report the median.

- Record exact commands and configuration values. Raw screenshots or large dumps without interpretation are not sufficient.

- All example outputs below are invented to illustrate required fields, not expected measurements. Your labels may differ if fields, units and phases remain identifiable. Counter names and available fields vary by kernel; record unavailable fields explicitly.

3.1 Required Capability Check

Record these checks before stress: uname -r; getconf PAGESIZE; nproc; free -h; swapon --show; findmnt -T ~/assignment9; readable /proc/pressure/memory and /proc/pressure/cpu; readable /proc/zoneinfo and required /proc/vmstat fields; sysctl vm.min_free_kbytes vm.watermark_scale_factor vm.watermark_boost_factor vm.swappiness; gcc --version; and collector operation. Record /proc/self/cgroup and any inherited memory.high, memory.max or memory.swap.max limits. Do not change cgroup limits to substitute for system memory pressure.

Under administrator privileges verify PSI trigger registration and temporary sysctl write/readback/restoration. Stop if these mandatory capabilities fail and contact the instructor; missing features are not zero measurements. For optional zram, record modinfo zram, device/module availability and compression algorithms; never reset or remove an existing device used by other work. Systemd or cgroup-controller configuration is not an implementation requirement.

Save original sysctl values and the active swap device list before testing. Prepare cleanup before starting any workload. Use a duration limit, explicit maximum memory, and a recovery terminal; stop allocation if headroom is exhausted. Restore only swap devices you changed. Run swapoff only while workloads are stopped and sufficient RAM is available; confirm success before a swap-off case. Do not run global configuration or cache operations alongside other WSL work.

References: https://learn.microsoft.com/en-us/windows/wsl/wsl-config ;

https://docs.kernel.org/accounting/psi.html ; https://docs.kernel.org/admin-guide/sysctl/vm.html

3.2 Procedure for Each Measurement Run

- Assign a case/run ID and record exact commands, settings and workload byte sizes. P is a sizing reference, not a program argument: if P = 3GiB, 0.5P = 1,610,612,736 bytes. Keep finalized sizes fixed across repeated comparisons.
- Apply the same cache/swap preparation, then start the collector, psimon polling and a separate probe process before the workload. Use a finite watchdog timeout. Aim for about 60 s per steady-state interval and record actual timing. Include the initial allocation in the Level 1 burst; distinguish fill, hold and retouch for swap experiments.
- Every student program must log CLOCK_MONOTONIC seconds at start and at each phase, probe window or event boundary. Match these times to the collector monotonic_s column. Independent elapsed times from separately started programs are not a common time origin.
- Save raw logs and exit status, select the interval when workload and monitors overlap, restore settings and repeat each configuration at least three times. Retain failed runs separately and explain exclusions. Report all valid runs, their median and spread.

```
mkdir -p runs
python3 student_support/sample.py runs/L1_C_01.csv --seconds 120
timeout --signal=INT 120s ./memhog probe > runs/L1_C_01_probe.log
timeout --signal=INT 120s ./psimon > runs/L1_C_01_psi.log
```

Run monitors in separate terminals before the workload; exclude idle lead-in/tail time. timeout status 124 is expected only for deliberately stopped monitors.

4. Required Tools and Test Programs

4.1 Student Tools and Instructor-Provided Sampler

```
./memhog <mode> [options]
./psimon [--interval <ms>] [--trigger <some|full> <stall-us> <window-us>]
# Level 3: extend memhog or add a C workload with documented options
python3 student_support/sample.py <output.csv> [--seconds <sec>]
```

Build memhog and psimon with your Makefile. A successful build must produce the executables used in the commands below. Implement the zoneinfo script and Level 3 C extensions yourself. Use the supplied sample.py; create its output directory first and choose a new CSV filename for each run (existing files are not overwritten).

4.2 Required memhog Modes

Mode	Purpose	Behavior
anon <size> [--rate <MiB/s>] [--hold <sec>]	allocate anonymous memory	map and write each actual OS page at the chosen rate, then hold; add full-page fill and seeded random retouch for swap experiments
file <path> [--rate <MiB/s>] [--loops <n>]	read file data into the page cache	read the file sequentially with read() in 1 MiB blocks; repeat for the given number of loops
ws <size> [--touches <n/s>] (optional)	keep a working set in use	touch random pages of an anonymous region of the given size at the given rate so that the pages stay recently used
probe [--interval <ms>] [--size <KiB>]	measure allocation latency	repeat: map a fresh region, touch every page, record the elapsed time, unmap; print p50, p99, and max every 5 seconds

A first touch of anonymous memory is a page fault that needs a free page, so probe latency includes any reclaim the allocation has to do. Run probe as a separate process during every experiment.

Required modes are anon, file and probe; ws is optional. Sizes accept integer bytes or KiB/MiB/GiB suffixes (1MiB = 1,048,576 bytes). --rate uses MiB/s, with 0 meaning no deliberate throttling. Reject unknown options, negative/invalid values and allocation/I/O errors with an explanatory stderr message and nonzero status.

- anon: write every mapped OS page; mmap alone is insufficient. Default --rate is 0 and --hold is 30 s. Apply the rate during initial page writes, hold the mapping for the requested time, then unmap and exit. Print phase, bytes/pages written and elapsed time.

- Level 2 anon extension: support --retouch random --seed <integer>. Fill whole pages with deterministic pseudo-random data, not a single repeated byte. After the hold phase, visit each page once in a seed-reproducible random order, read at least one byte and retain a checksum so reads are not optimized away. Report fill/hold/retouch separately.

- file: use buffered read() with a 1MiB buffer, handle short reads, rewind between passes and exit after --loops passes. Defaults: --loops 1 and --rate 0. Print file size, bytes read, elapsed seconds and MiB/s per pass and overall. Use a fully written, unchanged ext4 file, not a sparse file or O_DIRECT.

- probe: defaults are --interval 100 ms and --size 64 KiB. Time fresh mapping plus writes to each page using CLOCK_MONOTONIC; stop timing before unmapping and exclude sleep. Report sample count, p50/p99/max in microseconds every 5 s and a final partial window on SIGINT/SIGTERM. Use nearest-rank percentiles x[ceil(p*n)] on sorted samples.

Execution and output examples (invented values):

```
./memhog anon 256MiB --rate 64 --hold 10
phase=fill bytes=268435456 pages=65536 elapsed_s=4.08
phase=hold elapsed_s=10.00 status=ok
./memhog file data.bin --loops 2
pass=1 bytes=536870912 elapsed_s=2.50 throughput_MiB_s=204.80
./memhog probe --interval 100 --size 64
window_start_s=0.00 window_end_s=5.01 n=50 p50_us=18 p99_us=94 max_us=94
```

These abbreviated examples omit the common-clock timestamps required in Section 3.2. Probe measures allocation latency in its own process, not the workload request latency. State the probe window used in your table; do not average window p99 values and call the result an experiment-wide p99. A per-sample log and documented aggregation are also acceptable.

4.3 Required psimon Behavior

- Polling mode: default --interval is 1000 ms. Read memory some/full avg10 and total, compute interval stall percentages using measured elapsed time, and print both totals, percentages and units. The first interval is NA. A decreasing total invalidates that interval; do not print a negative percentage.

- Trigger mode: open /proc/pressure/memory with O_RDWR | O_NONBLOCK, write a trigger such as "some 150000 2000000", wait with poll() for POLLPRI, and print a timestamp for every event.

- If trigger registration fails, print the reason, close the descriptor and explicitly fall back to polling. Never report a permission error or rejected trigger as a successful registration. Handle SIGINT/SIGTERM and close descriptors on exit.

Polling and trigger examples (invented values; use a separate instance for each mode):

```
./psimon --interval 1000
elapsed_s=1.000 some_pct=4.00 full_pct=0.50 some_avg10=1.20
some_total_us=140000 full_total_us=25000
sudo ./psimon --trigger some 150000 2000000
registered metric=some stall_us=150000 window_us=2000000
event elapsed_s=12.480 metric=some
```

The polling example is wrapped for readability. Registration and event lines must be distinguishable. The trigger requests 150 ms of some stall in a 2 s window. Use sudo for this demonstration, validate the window/threshold and test one valid registration and one rejected request before pressure experiments. CPU PSI comes from sample.py; psimon needs no CPU mode.

5. Level 1 - Observe Watermarks, kswapd, and Direct Reclaim

5.1 Required Implementation

- Implement memhog with the anon, file, and probe modes.
- Write a shell/Python script that reads /proc/zoneinfo and prints one row per node/zone: node, zone, free_pages, min_pages, low_pages and high_pages. Also print MemFree and MemAvailable in KiB. Read live values rather than hard-coding zone names or watermarks.
- Before Level 1, stop all lab workloads, record the active swap devices, and disable swap with Linux administrator privileges. Verify /proc/swaps has no active entries; if disabling fails, stop and resolve it before running. Restore only the original swap configuration after the level. These operations affect the shared WSL VM, not just the current shell.
- Do not use more than one workload process per role (one memhog, one probe) in the required cases.

5.2 Required Functional Experiments

Demonstrate all of the following before the case studies:

Experiment	Required Action	What to Explain
Watermark values	print min, low, and high for every zone and compare them with vm.min_free_kbytes and vm.watermark_scale_factor	how min, low, and high relate; whether low - min equals high - low in each zone, and why a difference may appear (for example, watermark boost)
Per-zone free pages	show free pages per zone next to MemFree and MemAvailable	why total free memory is not what an allocation checks
Probe baseline	run probe on the idle VM	the allocation latency without reclaim, as a reference
Counter rates	run memhog anon with a small size and show that pgscan_* and allocstall stay near zero	what "no reclaim" looks like in the counters

5.3 Required Case Study

For each case below, prepare swap/cache state, start collector + psimon polling + probe, then start the pressure workload. Use enough file passes or hold time for the intended interval (about 60 s for a steady-state case); keep the initial burst inside case C measurements. Run A, B and C at least three times each. Sizes are starting points: pilot carefully, document adjustments, then freeze byte sizes and rates.

Case	Workload	Expected Reclaim Path
A - Idle	no stress	none
B - Background	memhog file with a file of 1.5P, read at a fixed rate (pilot example: 400MiB/s)	intended: mostly background reclaim; verify
C - Direct	warm the page cache with memhog file (1.0P), then memhog anon 0.9P at maximum rate	intended: background and direct reclaim; verify

For each case, report per-second rates and the probe latency:

Case	Free vs. Watermarks	pgscan_ kswapd/s	pgscan_ direct/s	allocstall /s	Probe p50 / p99	Interpretation
A						
B						
C						

Before running case C, predict which counters will rise and how probe p99 will change. State whether the measured result matched your prediction.

5.4 Baseline Record

This Level 1 result is the baseline for the later levels. Record at least:

- the watermark values of each zone and the sysctl values that produced them
- the three cases above, each repeated three times, with the median reported
- the sampled interval in which reclaim counters first increased in case C, with the corresponding per-zone free pages and watermarks; 1 s sampling cannot prove the exact instant or a crossing of min
- kswapd CPU use observed with top or pidstat during cases B and C

5.5 Required Explanation

1. Why can free memory be low on a healthy system, and why is MemAvailable a better estimate of headroom than MemFree?
2. In terms of watermarks, when does kswapd start, when does it stop, and when does a task start direct reclaim?
3. Why does direct reclaim raise allocation latency and tail latency, and which counters show that it happened?
4. Why must the allocation check use the zones a request can use instead of total free memory?

6. Level 2 - Diagnose Stalls with PSI, Thrashing, and Swap

6.1 Add PSI Monitoring

Add the required C psimon implementation and the anon random-retouch extension from Section 4.2. Run them with the supplied sample.py. Reuse the Level 1 tools and keep their measurement definitions unchanged.

Source	Signal	What It Tells Us
/proc/pressure/memory	some, full, avg10/60/300, total	how long tasks wait for memory
/proc/pressure/cpu	some	whether the slowdown is a CPU wait instead
/proc/vmstat	pgscan_*, allocstall_*, workingset_refault_file, workingset_refault_anon, pswpin, pswpout	which path reclaim takes and what is read back after eviction
PSI trigger	POLLPRI event	when the stall crosses a threshold, without polling

6.2 Required Case Study: What Is the Task Waiting For?

For cases 1-3, use the same reader looping over the same regular ext4 file of 0.5P. Case 4 instead measures the anonymous random-retouch workload and swap-in activity. Record memory and CPU PSI and counter rates, with probe running. Use swap off for cases 1-3 and the recorded disk swap for case 4. Adjust suggested load sizes in a pilot if needed, then freeze them for the comparison.

Case	Situation	How to Create It
1 - Healthy	file stays in the page cache	reader alone
2 - CPU contention	no memory shortage; CPUs are busy	reader plus busy-loop processes (for example, 3 per CPU)
3 - File thrashing	working set larger than the memory left for the cache	hold memhog anon 0.7P and run the same 0.5P file reader; pilot and freeze sizes
4 - Swap-in	pages come back from swap	enable swap; memhog anon 1.3P touched once, then touched again in random order

Case	Workload Speed / Time	memory some %	memory full %	cpu some %	refault_file /s	pswpin /s	pswpout /s	Interpretation
1								
2								
3								
4								

Compute stall percent = 100 x (end total - start total) / (elapsed seconds x 1,000,000). Show one calculation. some and full are separate overlapping measures, not values to add. avg10/60/300 use different averaging windows; explain their relationship to the measured interval without requiring numerical agreement.

For cases 1-3, report file-reader MiB/s. For case 4, report fill and retouch seconds separately. Do not treat these as the same performance unit. Each finalized case needs three runs; report the median and spread. Example PSI calculation: a 200,000 us increase in some total over 2 s means 10% some stall, not 200,000%.

6.3 Required Disk Swap Experiment and Optional Zram Comparison

Required: use the WSL disk swap and implement an anonymous workload with bounded duration, full-page deterministic nonzero data filling, and a reproducible random retouch pass (document options such as --retouch random and --seed). Use 1.3P only as a pilot starting point. Choose a working set that produces measurable swap activity while fitting RAM plus swap with headroom, then repeat three times with probe and the same byte sizes. Report elapsed time, actual swap rates, PSI and probe p99. If activity is small, redesign and record the evidence; do not increase memory without bounds.

Backend	Setup	Run Time	pswpin+pswpout /s	memory some %	Probe p99	CPU Use
Disk swap (required)	active WSL disk swap; record device, capacity and zswap state					
zram (optional)	supported zram device; identical workload; record capacity and algorithm					

Optional: if modinfo zram and a controlled module/device test succeed, compare disk swap with zram using the identical workload. Use one backend at a time, stop workloads before switching, record capacity, compression algorithm and zswap state, and restore original devices afterwards. Do not install a custom kernel for this assignment. If unsupported, record the reason; there is no deduction for omitting this optional comparison. Record page-fill compressibility; touching one byte in otherwise zero-filled pages is insufficient for this comparison.

Case 4 in Section 6.2 also satisfies the required disk-swap experiment here; no second identical set is required. Submit its phase timings, swap/PSI rates, probe latency, data-fill method and seed. The optional zram run uses that same workload for comparison.

6.4 Required PSI Trigger Experiment

- Start psimon with a trigger (for example, "some 150000 2000000", meaning 150 ms of stall within 2 s).
- Run cases 3 and 4 and record trigger registration, threshold/window, memory PSI and event times. If the threshold is not reached, use a justified valid lower threshold or redesign the bounded workload and repeat. Record any remaining absence of events honestly; an invalid trigger or permission error must not be reported as a successful pressure event.
- Explain how the trigger differs from polling in CPU use and in how quickly a program can react.

6.5 Required Explanation

5. What is the difference between some and full, and why can a slow task show 0 memory PSI?
6. Why is PSI more useful than memory usage for detecting a memory-related delay?
7. How do workingset_refault_file, pswpin, and pswpout distinguish file thrashing, a swap-in wait, and swap-out only?
8. Why can compressed swap in RAM be faster than disk swap, and what does it cost?

7. Level 3 - Design Workloads and Evaluate Memory Reclaim Tuning

Use the Level 2 psimon and the instructor-provided sampler to compare memory pressure before and after changing the sysctl variables on lecture slide 46. Designing and implementing a workload that exposes the effect of each setting is part of this level. Submit a report that explains both the observed benefit and its cost.

7.1 Required Settings and Hypotheses

Choose at least two sysctl variables from slide 46. For each, predict the affected counters, PSI and latency; justify an alternative to the actual original value read from WSL.

- vm.min_free_kbytes: free-page reserve. Example load: sustained memory use plus allocation bursts.
- vm.watermark_scale_factor: watermark spacing. Example load: a fixed working set plus periodic bursts.
- vm.swappiness: anonymous/file reclaim preference. Example load: anonymous memory plus buffered file reads with active swap.

Change one variable at a time; keep other settings, swap backend and cgroup limits fixed and recorded. Zram and memory.high/low are not sysctl choices. cgrun is not required.

7.2 Required Workload Implementation

- Extend memhog or add a C workload that exposes each chosen variable. Unexplained reuse of Level 1 loads is insufficient. Expose relevant sizes, access patterns, burst rates and duration; touch pages and bound runtime/memory. Supply build/run commands.
- The examples above are design hints, not required implementations or guaranteed outcomes. Use regular non-tmpfs files for buffered I/O and record data compressibility.
- Run probe separately; report allocation p50/p99/max and workload throughput or completion time. Define interval, sample count and percentile method; separate initialization/warm-up from measurement.

7.3 Controlled Comparison

- Pilot first: obtain measurable reclaim without aiming for OOM. Record workload bytes and fraction of baseline P; freeze bytes for paired runs. Do not re-size from a new P after tuning.
- Repeat original and changed settings at least three times each; report every run, median and spread. Alternate order or control its effects. Two variables x two values x three runs = 12 runs. A revised load needs both conditions rerun.
- Fix WSL environment/profile, workload, swap backend, warm-up, duration and background activity. Document cache and residual swap handling. Do not mistake cgroup-limited reclaim for a system watermark effect.
- Save originals; use temporary administrator sysctl writes and read back each value. Restore/verify originals before the next variable and at completion, including normal/error/interruption cleanup. Do not edit persistent sysctl files or use extreme min_free_kbytes values. Use the environment checked in Section 3.1.

7.4 Measurement and Result Table

Use the same interval and scope for psimon and raw sampler CSVs. Create one row per run, linked to raw filenames, with variable, original/effective value, run order, workload parameters, interval and interpretation. Include:

- Pressure/performance: memory some/full percentages from total differences; probe p50/p99/max (microseconds); workload throughput (MiB/s) or completion time (seconds).
- Reclaim/swap: pgscan_kswapd*, pgscan_direct*, pswpin/out (pages/s), and allocstall* (events/s).
- Memory state: MemAvailable, Cached and swap use with units. For watermark changes, include zone snapshots and kswapd CPU use over the same interval.

PSI % = 100 x delta total_us / (elapsed_s x 1,000,000). Do not add some and full. avg10 is smoothed and need not match the interval percentage; explain window differences. Sum only disjoint counters, never an aggregate plus components. Mark unavailable fields unsupported, not zero. Summarize repetitions and variability.

7.5 Workload Redesign When Effects Are Small

- Check applied values, actual page writes, pressure level, active swap for swappiness, and cgroup limits. Revise size, access pattern or bursts using that evidence; rerun original and changed settings under the same revised load.
- Submit initial/revised designs and results. Small effects are valid with a reasoned redesign attempt and environment limits explained. Do not discard unfavorable runs or present predictions as measurements; improvement in a particular direction is not required.

7.6 Required Report and Design Decision

- Report variable meanings, hypotheses, workload/method, results and a supported mechanism. Connect PSI, probe tail latency and reclaim counters; explain benefits, costs and uncertainty. Include exact commands, raw-data references and restoration evidence.
- For a small-memory webOS-class device, choose two tested settings to evaluate first. State each cost and verification signal. Treat this as a target-device validation proposal, not a product claim.

7.7 Required Analysis

- Why should this workload expose the variable, and which counters support the reclaim-path explanation? Why change only one setting?
- Did PSI and probe p99 change together? Explain differences and any memory, CPU, I/O or latency cost.
- What did workload redesign reveal? Use your results to justify the recommendation in Section 7.6.

8. Evidence Required

Level	Implementation Evidence	Runtime / Measurement Evidence	Analysis Evidence
1	memhog (anon, file, probe) + zoneinfo script	three reclaim cases with per-second rates and probe latency	watermarks, kswapd vs. direct reclaim, per-zone free pages
2	psimon (polling and trigger)	four wait cases, disk swap, trigger evidence; optional zram	some vs. full, refault vs. swap-in, compressed swap cost
3	C workload implementation and documented parameters	at least two sysctl comparisons, repeated runs, redesign evidence	workload sensitivity, PSI and latency, tuning costs and limitations

9. Submission Package

- Source code for memhog and psimon, the zoneinfo script, any additional Level 3 C workload, and a Makefile. Include the supplied student_support/sample.py and your experiment/analysis scripts. A clean make must build the executables used by your README commands.
- README with exact build/run commands, WSL configuration and preflight results, workload byte sizes, sysctl values, collector schema and units, measurement procedure and restoration evidence.
- Technical report (recommended 5-7 pages) organized as Level 1 / Level 2 / Level 3.
- Level 1 watermarks and three reclaim cases, including sampled per-zone state near observed reclaim activity and the limits of 1 s sampling.
- Level 2 four wait cases, the required disk-swap result, PSI trigger evidence and one interval percentage calculation. Include the optional zram comparison only if performed, otherwise state its support status.
- Level 3 baseline and changed-setting tables for at least two sysctl variables, workload design and redesign evidence, restoration records, and the small-memory-device design decision.
- A diagram of the reclaim path and the signals you read at each point (watermarks, kswapd, direct reclaim, /proc/vmstat, and PSI).
- The raw sampler CSV files and the scripts that produced the tables, so that results can be reproduced.
- A short AI-use statement if AI tools were used, including at least one generated suggestion or code issue that you verified, changed, or rejected.

10. Evaluation

Category	Weight	What Is Evaluated
Implementation	35%	correct memhog and psimon behavior, purposeful Level 3 workload implementation, and error handling
Experiment & Measurement	30%	controlled conditions, repeatability, rates computed from two samples, and the quality of the comparison
Technical Explanation	25%	correct reasoning about watermarks, reclaim paths, PSI, thrashing, swap, and sysctl tuning costs
Code Quality & Report	10%	clear structure, reproducibility, and concise evidence

Performance is not graded by absolute numbers across different hardware. The quality of the controlled comparison, the correctness of the measurement method, and the explanation of observed behavior are graded.

11. Final Check-Off Questions

- Show the watermarks of one zone and explain how they follow from vm.min_free_kbytes and vm.watermark_scale_factor.
- Explain why low MemFree alone does not mean that the system is under memory pressure.
- Before running case C, predict which counters will rise and how probe p99 will change, then compare with the measurement.
- Show a case in which a task is slow but memory PSI stays near 0, and explain what it waits for.
- Use pswpin, pswpout, and workingset_refault_file to tell a swap-in wait from file thrashing in your own data.
- Explain how your Level 3 workload makes the effect of each chosen sysctl variable observable.
- Compare a baseline and changed run using PSI, probe p99, and reclaim counters; explain the benefit and cost.
- Show your workload redesign, or justify why the initial design was sufficient, and verify that original settings were restored.

Build the tools. Stress the memory. Observe what changes. Explain why.